# Smart Restaurant Management System

## Project Idea Proposal - Phase 1

Graduation Project: Full Stack Application (Backend + Frontend) | 2026

## 1. Project Idea Description

Smart Restaurant Management System is a comprehensive digital platform designed to streamline daily restaurant operations. The system provides a secure, scalable solution for managing orders, users, menus, and inventory  replacing manual processes with an organized, role-driven digital workflow.

In Phase 1, the project focuses on building a robust Core Management System that includes:

- Secure JWT-based authentication system
- Role-Based Access Control (RBAC) for different staff types
- Full Menu Management (categories & products)
- Orders System with a clearly defined status workflow
- Inventory Management with automatic stock deduction
- Basic Dashboard with key performance indicators

The architecture is designed from the ground up to be extensible, allowing future integration of advanced analytics and AI-powered prediction layers without requiring a full system rebuild.

## 2. Roles & Permissions

The system implements a four-tier Role-Based Access Control (RBAC) model to ensure that each user type has clearly scoped capabilities:

| Role | Permissions |
|---|---|
| Admin (Owner) | Full system control: manage users, menu, inventory, and all reports |
| Manager | Manage products & categories, monitor orders, update inventory, view dashboard |
| Cashier | Create and close orders, view own orders, modify orders before preparation starts |
| Kitchen Staff | View incoming orders, update order status (Preparing / Ready) only |

Permissions are enforced via middleware on every protected route, ensuring no role can access functionality outside its scope.

# 3. Core Features (Phase 1)

## 3.1 Authentication & Authorization

- User registration with encrypted passwords (bcrypt)
- JWT-based login with token validation middleware
- Route protection and permission enforcement per role
- Soft-delete for users (no permanent deletion)

## 3.2 Menu Management

- CRUD operations for Categories (e.g., Beverages, Mains, Desserts)
- CRUD operations for Products with price, cost, and availability
- Auto-calculated profit margin (price − cost)
- Business rule: prevent deletion of products linked to past orders
- Business rule: prevent ordering unavailable products

## 3.3 Orders System (Core Business Logic)

- Create new orders with product selection and quantity
- Auto-calculated totals: Subtotal + Tax
- Status workflow enforced by role:
  - Pending → Preparing → Ready → Completed
  - Pending → Cancelled
- Cashier can modify orders only while status is Pending
- Inventory auto-deducted upon order confirmation
- Full order history with timestamps

## 3.4 Inventory Management

- Real-time quantity tracking per product
- Low-stock alerts when quantity falls below threshold (default: 5 units)
- Every stock movement logged in InventoryLog for full auditability
- System blocks order creation if stock is insufficient

## 3.5 Basic Dashboard

- Total daily sales revenue
- Total number of orders
- Top-selling products
- Order distribution by status

# 4. High-Level System Structure

## 4.1 Backend Stack

| Layer | Technology | Purpose |
|---|---|---|
| Runtime | Node.js | Server-side JavaScript execution |
| Framework | Express.js | HTTP routing and middleware pipeline |
| Database | MongoDB + Mongoose | Flexible document storage with schema validation |
| Authentication | JWT + bcrypt | Secure session management and password hashing |
| Validation | Custom Middleware (Manual Validation) | Input validation and sanitization logic |

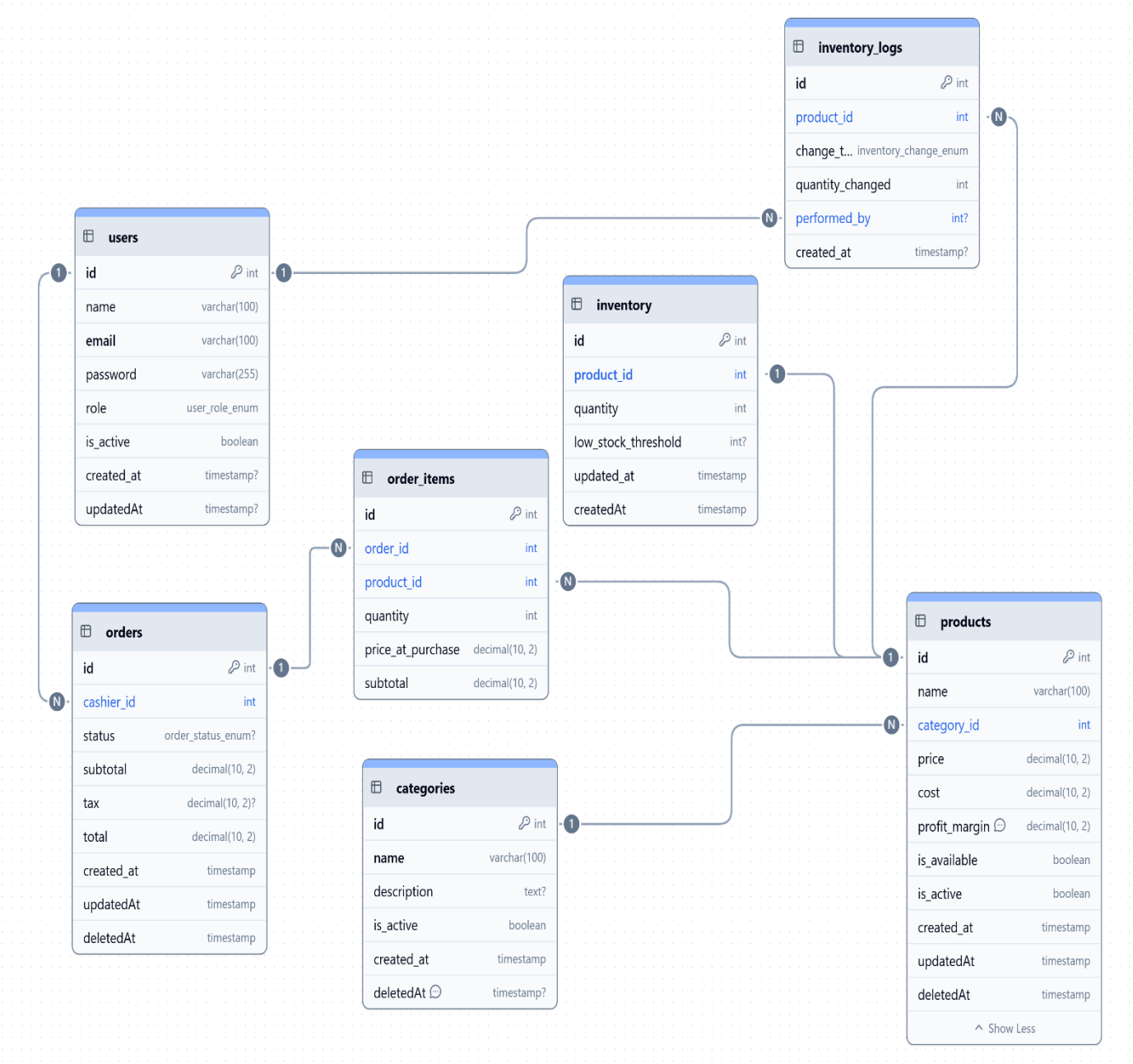
## 4.2 Database Design

The system uses MongoDB with 7 core collections:

| Collection | Key Relationships |
|---|---|
| Users | Referenced by Orders (cashier_id) and InventoryLog (performedBy) |
| Categories | One-to-Many with Products |
| Products | Many-to-One with Categories; One-to-One with Inventory; One-to-Many with OrderItems |
| Orders | Many-to-One with Users; One-to-Many with OrderItems |
| OrderItems | Many-to-One with Orders and Products (composite unique key) |
| Inventory | One-to-One with Products; auto-deducted on order confirmation |
| InventoryLog | Audit trail for all stock movements (restock / deduction) |

### 4.2.1 Entity Relationship Diagram (ERD)

The following diagram illustrates the full database schema and entity relationships for Phase 1:



# 5. Development Plan

| Phase | Focus Area | Deliverables |
|---|---|---|
| Phase 1.1 | Authentication & Users | Secure login system, JWT middleware, RBAC enforcement |
| Phase 1.2 | Menu & Categories | Full CRUD for categories and products, validation rules |
| Phase 1.3 | Orders System | Order workflow, total calculation, inventory deduction on confirmation |
| Phase 1.4 | Inventory | Quantity tracking, low-stock alerts, movement logging |
| Phase 1.5 | Dashboard | MongoDB Aggregation pipelines, KPI endpoints, frontend charts |
| Phase 2 (Future) | Advanced Analytics | Peak-hour analysis, profit margin reports, staff performance |
| Phase 3 (Future) | AI Prediction | Demand forecasting, smart restocking, sales prediction |

Each phase is designed as an independent deliverable, enabling incremental testing and demonstration throughout the development cycle.

# 6. API Structure (Phase 1)

The following REST API endpoints define the full backend interface for Phase 1. All protected routes require a valid JWT token and are enforced by role-based middleware

## 6.1 Authentication & Users

*Security foundation: JWT-based authentication with role enforcement on every protected route.*

| Method | Endpoint | Description | Access |
|---|---|---|---|
| POST | /api/auth/register | Register a new user (admin / manager / cashier / kitchen) | Admin |
| POST | /api/auth/login | Login and receive JWT token | Public |
| GET | /api/auth/me | Get current authenticated user's data | All roles |
| PUT | /api/auth/profile | Update own profile information | All roles |
| POST | /api/auth/change-password | Change own password | All roles |
| GET | /api/users | List all system users | Admin |
| PUT | /api/users/:id/role | Update a user's role | Admin |
| DELETE | /api/users/:id | Deactivate a user account (soft delete) | Admin |

## 6.2 Categories

*Prevents deletion of categories that contain active products.*

| Method | Endpoint | Description | Access |
|---|---|---|---|
| GET | /api/categories | List all categories | All roles |
| GET | /api/categories/:id | Get category details | All roles |
| POST | /api/categories | Create a new category | Admin / Manager |
| PUT | /api/categories/:id | Update category information | Admin / Manager |
| DELETE | /api/categories/:id | Soft-delete category (blocked if has active products) | Admin / Manager |

## 6.3 Products (Menu Items)

*Supports filters by category, price range, and availability. Profit margin is auto calculated.*

| Method | Endpoint | Description | Access |
|---|---|---|---|
| GET | /api/products | List products (filter: category, price, availability) | All roles |
| GET | /api/products/:id | Get product details | All roles |
| POST | /api/products | Create a new product | Admin / Manager |
| PUT | /api/products/:id | Update product data | Admin / Manager |
| DELETE | /api/products/:id | Soft-delete product (blocked if linked to orders) | Admin / Manager |
| PATCH | /api/products/:id/status | Toggle availability (in-stock / out-of-stock) | Admin / Manager |

**Business Logic:**

- Prevent selling unavailable products
- Auto-calculate profit margin (price − cost)
- Block deletion of products linked to past orders

## 6.4 Orders (Core Business Logic)

*The heart of the system manages the full order lifecycle with strict status transitions.*

| Method | Endpoint | Description | Access |
|---|---|---|---|
| GET | /api/orders | List all orders (filter: date, cashier, status) | Admin / Manager |
| GET | /api/orders/:id | Get order details with items | Admin / Manager / Cashier |
| POST | /api/orders | Create a new order | Cashier |
| PUT | /api/orders/:id | Modify order (Pending status only) | Cashier |
| PATCH | /api/orders/:id/status | Update order status | Kitchen / Cashier |
| DELETE | /api/orders/:id | Cancel an order | Cashier / Admin |

**Order Status Workflow:**

| Transition | Allowed By | Rule |
|---|---|---|
| Pending → Preparing | Kitchen Staff | Locks order from further edits |
| Preparing → Ready | Kitchen Staff | Notifies cashier for pickup |
| Ready → Completed | Cashier | Closes the order |
| Pending → Cancelled | Cashier / Admin | Only before preparation begins |

**Business Logic:**

- Inventory is auto deducted when an order is confirmed
- Orders cannot be edited once status moves to Preparing
- Kitchen staff can only update status; cannot modify order contents

## 6.5 Inventory Management

*Tracks real-time stock levels and logs every movement for full auditability.*

| Method | Endpoint | Description | Access |
|---|---|---|---|
| GET | /api/inventory | View full inventory list | Admin / Manager |
| GET | /api/inventory/:productId | Get stock details for a product | Admin / Manager |
| PUT | /api/inventory/:productId | Update product quantity | Admin / Manager |
| POST | /api/inventory/restock | Add new stock (restock operation) | Admin / Manager |
| GET | /api/inventory/low-stock | List products below stock threshold | Admin / Manager |
| GET | /api/inventory/logs | View full inventory movement history | Admin / Manager |

**Business Logic:**

- Order creation is blocked if requested quantity exceeds available stock
- Every stock deduction and restock is recorded in InventoryLog with timestamp and performed-by reference